PRJ_221: Behavioural Change Monitoring Software

Final Project Review, Live Demo Guide & Comprehensive Reviewer Defense Sheet

Candidate Role: Lead Developer & Researcher	**Tech Stack:** React 19, FastAPI, PostgreSQL, Scikit-Learn
Target Audience: Academic Reviewers & Evaluators	**Status:** All Development & Security Phases Verified (100%)

1. Complete Project Status: Done vs. Optional Future Scope

Every single functional, architectural, security, testing, and deployment phase required for your software is **100% completed and verified**.

Phase	Component / Milestone	Status	Verification Details
Phase A	New Assessment & Prediction Pipeline	DONE	POST /predict & /behavior, 44 features, RF + Isolation Forest
Phase B	Authentication & Cryptographic Security	DONE	JWT (8h expiry), PBKDF2 (310k iter), Anti-IDOR, /auth/me
Phase C	Role-Based Access Control (RBAC)	DONE	Strict Student vs. Faculty views, 401 & 403 HTTP enforcement
Phase D	Faculty Cohort Analytics & Alerts	DONE	145+ cohort overview, student filters, drill-down modal, alerts queue
Phase E	Automated Testing Suite (20 Tests)	DONE	12 API tests, 5 ML tests, 2 DB tests, 1 E2E test, 0 lint errors
Phase F	Project Cleanup & Dependencies	DONE	Clean requirements.txt, .gitignore, health_check.py, seed_database.py
Phase G	Cloud & Deployment Architecture	DONE	init_db.sql, Dockerfile (backend/frontend), docker-compose, AWS guides
Phase H	Academic Deliverables (External)	OPTIONAL	Physical printed thesis book / slides (Provided in this Guide!)

2. Explain the Project Like I'm 5 Years Old

The Kid Story: Imagine you have a special Smart School Watch. Every time you open a book, watch a video lesson, ask a question in class, or do your homework, your watch notices.

Normally, teachers only find out if a student is struggling after they fail an exam at the very end of the year. By then, it's too late!

Our software is like a **Super-Friendly Health Tracker for Learning**:

1. **The Brain Watcher (Random Forest):** It looks at the student's study patterns over 8 classes. It says: *'Hey, this student is doing great and Improving!'* or *'Uh oh, this student stopped watching lectures and is Declining.'*
2. **The Weird-Habit Spotter (Isolation Forest):** If a student usually studies 1 hour a day, but suddenly studies 12 hours at 3 AM or 0 minutes all week, it rings a bell: *'Hey teacher, something unusual is happening!'*
3. **The Magic Magnifying Glass (SHAP Explainable AI):** Instead of being a secret magic box, it tells the student: *'You're improving because you contributed to the forum +30% more this month!'*
4. **The Fort Knox Lock (Security):** Nobody can peek at another student's diary! A student can ONLY see their own study health, while the teacher can see everyone to help whoever needs help.

3. Step-by-Step Live Demo Script for Your Final Review

Follow these exact steps during your presentation. This flow highlights your engineering depth, ML rigor, and security safeguards.

Step 1: System Readiness & Tests (Terminal Proof)

Before opening the browser, show your terminal.
Command: .venv\Scripts\python.exe scripts/health_check.py
Show the Reviewer: Point out that all 4 ML models are loaded, PostgreSQL 18 is connected, and 23 REST endpoints are verified.
Then Run: .venv\Scripts\python.exe -m unittest discover tests
What to Say: *'We have a 100% automated test suite with 20 unit and integration tests verifying ML predictions, RBAC, anti-IDOR security, and database constraints in 1.2 seconds.'*

Step 2: Student Login & Real-Time Dashboard

Open http://localhost:5173.
Click the blue preset button: **'Login as Student'** (00050f0e@example.com).
Show the Reviewer:
• **Behavioral State Badge:** Shows 'Improving' / 'Stable' / 'Declining' predicted by Random Forest.
• **Behavioral Change Index (BCI):** Standardized metric across 8 sequential periods.
• **Pattern Anomaly Score:** Isolation Forest anomaly detector score.
• **Interactive Trend Chart:** Switch dropdown between 'Session Count', 'Lecture Views', and 'Forum Contributions' to show how activity evolved from Sequence 1 to 8.

Step 3: Explainable AI (SHAP Insights)

Click the **'Insights'** tab on the left sidebar.
Show the Reviewer: The horizontal SHAP feature importance bars.
What to Say: *'Our system solves the black-box problem of machine learning using TreeSHAP. Instead of just giving a prediction, it mathematically attributes the exact features—such as forum contribution trend and video regularity—that pushed this student into the Stable or Improving state.'*

Step 4: Anti-IDOR Security Proof (Crucial Defense Point)

Open browser Developer Tools (F12) -> Console or Network tab.
Explain to the reviewer: *'A common security flaw in student portals is IDOR (Insecure Direct Object Reference), where Student A changes the URL ID to Student B and views their confidential grades.'*
Show that if an unauthorized ID is queried, the FastAPI backend verifies the signed JWT sub claim and immediately denies the request with an **HTTP 403 Forbidden** error. The backend never blindly trusts the frontend!

Step 5: Role Switch & Faculty Cohort Analytics

Click **'Sign Out'**, then click **'Login as Faculty'** (testfaculty@example.com).
Show the Reviewer:
• **Cohort Summary:** Total student distribution (Improving, Stable, Declining, Elevated).
• **Live Filtering:** Search for a student name or filter by 'Declining' + 'High Priority'.
• **Student Drill-Down:** Click on any student row to open the complete longitudinal history, anomaly metrics, and rule-based intervention recommendations.
• **Faculty Alerts Queue:** View all high-risk students in one centralized screen.

Step 6: On-Demand New Student Assessment Simulator

Click **'New Assessment'** in the faculty navigation.
Walk through Sequences 1 through 7 and click **'Generate Prediction'**.
What to Say: *'Faculty can simulate or assess new incoming students who have 7 sequence periods. The backend transforms their raw activity into 44 statistical features and instantly predicts their trajectory.'*

4. Technical Deep-Dive: Machine Learning & Algorithms

Be prepared to answer technical questions regarding your models, mathematical formulation, and architecture.

1. Dataset & Structure

1,136 longitudinal engagement observations across 142 simulated students over 8 distinct course sequences. Each sequence tracks 11 behavioral variables (frequencies, regularity indices, session counts, duration, and final grades).

2. Feature Engineering (44 Features)

For each of the 11 behavioral variables, the pipeline calculates 4 statistical dimensions across historical sequences 1 to 7:

- **Mean (μ):** Central tendency of engagement.
- **Standard Deviation (σ):** Volatility/consistency in student activity.
- **Recent Value (v_7):** Latest observation level.
- **Linear Trend Slope (m):** First-order polynomial regression coefficient measuring rate of change.

3. Behavioral Change Index (BCI)

Standardized change metric quantifying trajectory between baseline Sequence 1 and terminal Sequence 8:
BCI_i = (1 / K) * ∑ [(X_{i,8,k} - X_{i,1,k}) / StdDev(X_k)]

- **Declining:** BCI < 25th percentile
- **Stable:** 25th percentile ≤ BCI ≤ 75th percentile
- **Improving:** BCI > 75th percentile

4. Predictive Classification Model

Random Forest Classifier (trained with 100 estimators, balanced class weights, Gini impurity criterion). Chosen over deep neural networks due to its superior performance on tabular structured data, resistance to overfitting, and native compatibility with TreeSHAP.

5. Unsupervised Anomaly Detection

Isolation Forest with standard feature scaling. Employs recursive random space partitioning via isolation trees (iTrees). Outliers isolate with significantly shorter path lengths: s(x, n) = 2^(-E(h(x)) / c(n)). Output: 1 for normal activity, -1 for anomalous drop-offs or erratic spikes.

6. Explainable AI (TreeSHAP)

Implements Lundberg & Lee's TreeSHAP algorithm (O(TLD^2) polynomial time complexity). Attributes additive importance values ϕ_i where: f(x) = ϕ_0 + ∑ ϕ_i. Explains exactly which behavioral shifts drove the predicted state.

5. Security & System Architecture Deep Dive

Authentication & Password Storage

Passwords are never stored in plaintext. We utilize **PBKDF2-HMAC-SHA256** with **310,000 iterations** and a random 16-byte cryptographic salt (secrets.token_hex(16)). Password comparison uses constant-time digest verification (secrets.compare_digest) to eliminate side-channel timing attacks.

Stateless JWT Token Architecture

Upon successful authentication, the server signs an HMAC-SHA256 JWT containing sub (User ID), role (Student/Faculty), and an 8-hour expiration timestamp. Tokens are stored in browser localStorage and transmitted via HTTP headers: Authorization: Bearer <token>.

Anti-IDOR (Insecure Direct Object Reference) Protection

The backend never trusts client-supplied URL IDs. The reusable FastAPI dependency authorize_user_path() extracts the verified User ID from the decrypted JWT. If a student attempts to access another student's dashboard, an immediate HTTP 403 Forbidden is returned.

Relational Database Schema (PostgreSQL)

Consists of 6 normalized relational tables: users, behaviour_records, predictions, anomalies, recommendations, and alerts with strict foreign keys (ON DELETE CASCADE) and unique composite indexes (uq_user_sequence).

6. Reviewer Viva Q&A;: 20 Questions & High-Scoring Answers

Here are the most common and difficult questions examiners ask in final defenses, along with direct, high-scoring responses:

Q1: What is the main problem this project solves?

Traditional academic analytics rely on summative assessments (final grades), which only flag student failure after the semester ends. Our software performs longitudinal, formative monitoring across sequential learning periods, predicting behavioral state transitions and detecting anomalous drop-offs in real time so faculty can intervene before failure occurs.

Q2: Why did you choose Random Forest over Deep Learning (LSTM or Transformers)?

Our dataset consists of tabular, sequence-aggregated behavioral metrics across 142 students. Deep learning architectures like LSTMs suffer from high sample complexity and prone to severe overfitting on small-to-medium tabular cohorts. Random Forest handles non-linear interactions, provides low variance, requires minimal hyperparameter tuning, and integrates natively with exact TreeSHAP algorithms for explainability.

Q3: What is BCI and how did you formulate it?

BCI stands for Behavioral Change Index. It calculates the normalized average delta between a student's baseline activity in Sequence 1 and their performance in Sequence 8 across all 11 behavioral features, standardized by the cohort standard deviation. It provides an objective, continuous ground truth to categorize trajectories into Declining, Stable, and Improving.

Q4: How does Isolation Forest detect anomalies?

Isolation Forest is an unsupervised tree-based ensemble. Instead of modeling normal clusters like k-means or DBSCAN, it isolates observations by randomly selecting a feature and split value. Because anomalies have extreme or rare values, they require fewer random recursive partitions to isolate, resulting in significantly shorter path lengths in the isolation trees.

Q5: What is the difference between SHAP and simple feature importance?

Global feature importance (like Mean Decrease in Impurity) only tells you what features were important across the entire model on average, without indicating directionality. SHAP (Shapley Additive Explanations), rooted in cooperative game theory, provides local explanations for each individual student, showing whether a specific feature (e.g., video watch frequency) increased or decreased their risk score and by how much.

Q6: How does the system prevent a student from seeing someone else's grades?

We implemented Anti-IDOR protection at the API gateway layer using FastAPI dependencies. When a student logs in, their identity and role are encoded in a signed JWT. On every protected request, the backend extracts the user ID from the token claims and verifies it matches the requested URL parameter. If it does not, an HTTP 403 Forbidden is raised.

Q7: How are passwords stored and secured?

Passwords are never stored in plaintext. They are salted with a 16-byte random hex string and hashed using PBKDF2-HMAC-SHA256 with 310,000 iterations. Verification uses secrets.compare_digest() to execute constant-time comparisons, preventing timing side-channel attacks.

Q8: Why did you use JWT instead of traditional server-side cookies/sessions?

JWT is stateless, eliminating server-side session memory overhead and allowing horizontal scaling across containerized microservices (such as AWS App Runner or ECS tasks). The token is cryptographically verified on each worker without needing a central session cache like Redis.

Q9: What happens when a user's token expires?

Tokens expire in 8 hours. When an expired token is presented, FastAPI raises an HTTP 401 Unauthorized with 'Token has expired'. Our frontend apiFetch wrapper intercepts the 401 response, automatically purges stale tokens from localStorage, and redirects the user to the login portal.

Q10: Why did you choose PostgreSQL over MongoDB / NoSQL?

Student behavioral records require strict relational integrity, ACID compliance, and foreign key constraints (e.g., student deletion must cascade to predictions and records). PostgreSQL also offers superior LATERAL join capabilities, allowing the faculty dashboard to query latest predictions, anomalies, and alerts in a single query.

Q11: Can a regular user register as a Faculty member?

No. Self-registration via POST /auth/signup always hardcodes the role to 'Student'. Faculty registration requires calling POST /auth/signup/faculty with a cryptographically verified server-side FACULTY_INVITE_CODE. If the code is missing or incorrect, an HTTP 403 Forbidden is returned.

Q12: How do you handle cold-start students who have no historical data?

For brand new students with zero historical records, our system provides an on-demand assessment simulator ('New Assessment') allowing faculty to enter 7 sequential periods. The backend transforms these into 44 features and runs live inference without requiring prior database history.

Q13: How did you validate that your software is bug-free and production-ready?

We built an automated test suite containing 20 unit and integration tests covering password hashing, JWT decoding, RBAC, anti-IDOR checks, ML inference, and PostgreSQL schema constraints. We also achieved zero ESLint errors and clean Vite production builds.

Q14: How is the application deployed to production?

The project is fully containerized using multi-stage Dockerfiles. The backend runs on Python 3.11 with Uvicorn workers, and the frontend is compiled into static assets served via an alpine Nginx reverse proxy with gzip compression and security headers. A multi-container docker-compose orchestrates PostgreSQL, FastAPI, and React.

Q15: What are the primary risk categories in your alert system?

Alerts are rule-based: High Priority is triggered when a student is predicted as Declining and flagged as an Anomaly. Medium Priority is assigned when a student is Declining but normal, or Stable but anomalous. Low Priority applies to Stable or Improving students with normal engagement.